

ESTRUCTURA DEL PROGRAMA DE NEWTON RAPHSON

```
Método de Newton Raphson.py ×
1 from sympy import symbols, log, diff, sin, cos
2 from sympy.parsing.sympy_parser import parse_expr
3
4 ERROR = 1E-6
5
6 x = symbols('x')
7
8 func = parse_expr(input('f(x): '))
9 x_0 = float(input('x_0: '))
10 dec = diff(func, x)
11
12 contador = 0
13 fx_1 = 100
14
15 while abs(fx_1) >= ERROR:
16     fx_0 = func.subs(x, x_0).evalf()
17     decx_0 = dec.subs(x, x_0).evalf()
18     x_1 = (x_0) - (fx_0 / decx_0)
19     fx_1 = func.subs(x, x_1).evalf()
20     e = abs((x_1 - x_0) / x_1)
21     print('iteración:', contador)
22     print('x_0:', x_0)
23     print('fx_0:', fx_0)
24     print('x_1:', x_1)
25     print('fx_1:', fx_1)
26     print('*' * 15)
27     if e > ERROR:
28         x_0 = x_1
29     contador += 1
30 print('Raíz: ', x_1)
31 print('f(Raíz):', fx_1)
```

Importacion de librerias

```
Método de Newton Raphson.py ×
1 from sympy import symbols, log, diff, sin, cos
2 from sympy.parsing.sympy_parser import parse_expr
3
```

sympy: Es una librería en Python que permite trabajar con matemáticas simbólicas. Se usa para realizar cálculos algebraicos, derivadas, integrales, etc.

symbols: Se utiliza para definir símbolos matemáticos (en este caso, x), los cuales representan variables.

log, sin, cos: Son funciones matemáticas, que permiten trabajar con logaritmos, seno, y coseno, respectivamente.

diff: Esta función se usa para calcular la derivada de una función simbólica.

parse_expr: Esta función permite convertir una cadena de texto en una expresión matemática simbólica. Es útil cuando se desea que el usuario ingrese la función.

Definicion de constantes y variables simbolicas

```
4 ERROR = 1E-6
5
6 x = symbols('x')
```

ERROR: Es el valor umbral que indica cuán cerca debe estar la aproximación de la raíz para considerar que hemos encontrado la solución. En este caso, se ha fijado como 1×10^{-6}

`x = symbols('x')`: Define el símbolo x como una variable simbólica que se usará en la función matemática.

Entrada de datos

```
8 func = parse_expr(input('f(x): '))
9 x_0 = float(input('x_0: '))
10 dec = diff(func, x)
11
```

func: Aquí el programa pide al usuario que ingrese una función, por ejemplo: $x^2 - 4$. La función ingresada por el usuario se convierte en una expresión simbólica mediante `parse_expr`.

`X_0`: Es el valor inicial de `x_0` que es necesario para comenzar el método iterativo. El valor que ingresa el usuario se convierte a tipo `float`.

Cálculo de la derivada

```
10 dec = diff(func, x)
11
```

`dec = diff(func, x)`: Calcula la derivada de la función `func` con respecto a la variable `x`. Esta derivada es necesaria para aplicar el método de Newton-Raphson.

Inicialización de las variables

```
12 contador = 0
13 fx_1 = 100
14
```

`contador = 0`: Es el contador que se usa para llevar el control de las iteraciones.

`fx_1 = 100`: Esta es una variable que se usa para almacenar el valor de la función en el valor de `x_1` durante las iteraciones. Se inicializa con un valor arbitrario para asegurar que el bucle comience.

Bucle del método de newton Raphson

```
15 while abs(fx_1) >= ERROR:
```

El bucle while continúa iterando mientras el valor absoluto de $f(x_1)$ sea mayor o igual al umbral de error.

Cálculo de los valores en la iteración

```
14  
15 while abs(fx_1) >= ERROR:  
16     fx_0 = func.subs(x, x_0).evalf()  
17     decx_0 = dec.subs(x, x_0).evalf()
```

Cálculo de la siguiente aproximación de la raíz

```
18     x_1 = (x_0) - (fx_0 / decx_0)
```

Evaluación de la aproximación en la nueva aproximación

```
19     fx_1 = func.subs(x, x_1).evalf()
```

Calcula el valor de la función en el nuevo valor x_1

Cálculo del error relativo

```
20     e = abs((x_1 - x_0) / x_1)
```

Este es el error relativo que nos indica cuán cerca estamos de la raíz. Si el error es mayor que el umbral ERROR, el ciclo continúa.

Impresión de resultados y actualización de valores

```
21     print('iteración:', contador)  
22     print('x_0:', x_0)  
23     print('fx_0:', fx_0)  
24     print('x_1:', x_1)  
25     print('fx_1:', fx_1)  
26     print('*' * 15)  
27     if e > ERROR:  
28         x_0 = x_1  
29         contador += 1
```

Aquí se imprimen los resultados de la iteración: el valor de x_0 , el valor de la función en x_0 , el nuevo valor x_1 , y el valor de la función en x_1 .

Si el error e es mayor que el umbral de error, se actualiza x_0 a x_1 para la siguiente iteración.

Se incrementa el contador para la siguiente iteración.

Resultado final

```
30 print('Raíz: ', x_1)  
31 print('f(Raíz):', fx_1)  
32
```

Después de que el ciclo termina, se imprime el valor final de la raíz x_1 y el valor de la función en esa raíz.